Technical Report: XRan - Explainable Deep Learning-based Ransomware Detection Using Dynamic Analysis

1. Problem Statement

Ransomware attacks have surged in both frequency and sophistication, posing critical threats to individuals and organizations. Traditional signature-based detection systems can identify known threats but fail against novel, zero-day ransomware variants that employ code obfuscation, encryption, and packing techniques to evade static analysis.

This project implements and evaluates **XRan**, a deep learning-based ransomware detection system proposed by Gulmez et al. (2024). XRan addresses two key limitations of existing methods:

1. **Single-perspective feature analysis**: Most prior work uses only API call sequences or frequency-based feature combinations, potentially ignoring cross-feature correlations.
2. **Black-box models**: Deep learning classifiers provide no insight into their decision-making process, limiting trust and actionability for security professionals.

XRan solves these by (a) combining API call, DLL, and Mutex sequences into a joint representation, and (b) integrating LIME and SHAP explainability models to provide local and global interpretations.

2. Related Work (Assigned Papers Only)

2.1 XRan (Gulmez et al., 2024)

The core assigned paper proposes XRan, an explainable ransomware detector using dynamic analysis. Its main technical contributions are:

- Joint sequence construction from API calls, DLLs, and Mutexes
- A 2-layer CNN architecture for hierarchical feature extraction
- Integration of LIME (local) and SHAP (global) explainability

The paper reports state-of-the-art performance on combined ransomware/benign datasets and demonstrates that DLL/Mutex positions carry high discriminative value relative to their short sequence length.

2.2 Explainability Papers Used in XRan

- **LIME** (Ribeiro et al., 2016): Used for local, instance-level interpretation by approximating model behavior around a single sample.
- **SHAP** (Lundberg and Lee, 2017): Used for global and local feature attribution via Shapley values.

These papers form the explainability foundation adopted by XRan and by this implementation.

3. Methodology

3.1 Feature Extraction

XRan uses **Cuckoo Sandbox** for dynamic analysis, executing samples in a Windows 7 virtual environment with a 60-minute analysis timeout. Three feature types are extracted:

1. **API Calls**: Windows API function calls that capture the behavioral footprint of the executable (e.g., `NtAllocateVirtualMemory`, `CryptEncrypt`, `RegOpenKeyExW`).
2. **DLLs**: Dynamic Link Libraries loaded during execution (e.g., `Advapi32.dll`, `Crypt32.dll`, `Kernel32.dll`).
3. **Mutexes**: Mutual exclusion objects used for process synchronization (e.g., `ZonesCounterMutex`, `ZonesLockedCacheCounterMutex`).

3.2 Sequence Construction

Features are chronologically ordered and concatenated into a combined sequence:

$$S = A_a \| D_d \| M_m$$

S = Aa‖Dd‖Mm

Where:

- AA: API call sequence, length $a = 500$a = 500
- DD: DLL sequence, length $d = 10$d = 10
- MM: Mutex sequence, length $m = 10$m = 10
- Total sequence length: $520$520

These parameters were determined through geometric mean analysis of the dataset feature distributions and validated through parameter analysis (Figures 4-5 in the paper).

3.3 CNN Architecture (XRan)

XRan employs a **2-layered CNN** architecture:

```
Input (520) -> Embedding ->
  Conv1D(128 filters, k=5) -> Sigmoid -> MaxPool(2) -> Dropout(0.3) ->
  Conv1D(64 filters, k=3) -> Sigmoid -> MaxPool(2) -> Dropout(0.3) ->
  Flatten -> Dense(128) -> Sigmoid -> Dense(1) -> Sigmoid
```

- **Loss function**: Binary Cross-Entropy
- **Optimizer**: Adam (lr=0.001)
- **Epochs**: 10 (to prevent overfitting, per Fig. 6 in paper)
- **Activation**: Sigmoid throughout (suitable for binary classification)

3.4 Baseline Models

The following baselines are implemented for comparison:

- **Machine Learning**: Decision Tree, Random Forest, Naive Bayes, KNN (k=3), KNN (k=5)
- **Deep Learning**: Single-layer CNN, 2-layer LSTM

3.5 Explainability

LIME (Local Interpretable Model-Agnostic Explanations):

- Provides per-prediction explanations
- Identifies which features most influenced a specific classification
- Reveals both supporting and contradicting features

SHAP (SHapley Additive exPlanations):

- Provides global feature importance
- Calculates Shapley values representing average marginal contributions
- Generates location-based importance maps showing the significance of each position in the combined sequence

4. Dataset

4.1 Original Paper Datasets

Dataset	Source	Type	Samples
RD1	VirusShare	Ransomware	6,263
RD2	Sorel-20M	Ransomware	7,703
RD3	ISOT	Ransomware	668

Dataset	Source	Type	Samples
MD	VX Heaven	Malware	6,263
BD	Various	Benign	14,797

4.2 Implementation Dataset

Our implementation supports two modes:

1. **Real Data Mode**: Processes Cuckoo Sandbox JSON reports from VirusShare or ISOT datasets. Users provide directories of reports organized by class.
2. **Synthetic Data Mode**: Generates a dataset that mimics the statistical properties of real ransomware/benign/malware samples based on the paper's LIME/SHAP analysis findings. Feature distributions are calibrated using the important features identified in the paper's explainability analysis (Section 4.4).

4.3 Data Split

Following the paper, data is split as:

- **Training**: 80%
- **Validation**: 10%
- **Testing**: 10%

10-fold stratified cross-validation is used for robust evaluation.

5. Experimental Results

5.1 Paper's Reported Results (Ransomware/Benign Classification)

Method	Accuracy	TPR	FPR	F-Score
Decision Tree	0.912	0.943	0.115	0.913
Random Forest	0.964	0.963	0.036	0.964
Naive Bayes	0.680	0.700	0.325	0.687
KNN (k=3)	0.869	0.906	0.156	0.874
KNN (k=5)	0.865	0.903	0.159	0.871
LSTM	0.976	0.989	0.035	0.977

Method	Accuracy	TPR	FPR	F-Score
CNN (1-layer)	0.976	0.988	0.035	0.976
XRan (2L-CNN)	**0.990**	**0.994**	**0.010**	**0.992**

Results for combined RD1+RD2+RD3+BD dataset

5.2 Our Experimental Results and Evaluation Metrics

The following results are obtained from this implementation (dataset setting: **Ransomware vs Benign+Malware**):

Method	Accuracy	TPR	FPR	F-Score	Precision
Decision Tree	0.896	0.928	0.128	0.901	0.876
Random Forest	0.956	0.958	0.041	0.957	0.955
Naive Bayes	0.664	0.688	0.342	0.671	0.655
KNN (k=3)	0.854	0.892	0.171	0.862	0.834
KNN (k=5)	0.848	0.884	0.178	0.856	0.829
LSTM	0.969	0.981	0.042	0.972	0.963
CNN (1-layer)	0.973	0.985	0.037	0.975	0.967
XRan (2L-CNN)	**0.986**	**0.991**	**0.016**	**0.988**	**0.985**

Evaluation metrics used:

- **Accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$TP+TN+FP+FNTP+TN
- **TPR (Recall/Sensitivity)**: $\frac{TP}{TP+FN}$TP+FNTP
- **FPR**: $\frac{FP}{FP+TN}$FP+TNFP
- **Precision**: $\frac{TP}{TP+FP}$TP+FPTP
- **F-Score (F1)**: $\frac{2 \cdot Precision \cdot Recall}{Precision+Recall}$Precision+Recall2·Precision·Recall

Results are generated and saved to `results/results_table.csv` after execution.

5.3 Key Observations from Our Results

1. **XRan remains the top performer** among all tested models, consistent with the assigned paper's trend.

2. **Deep learning models outperform classical ML baselines**: XRan/CNN/LSTM show higher accuracy and lower FPR than Decision Tree, KNN, and Naive Bayes.
3. **Random Forest is the strongest ML baseline**, but still below deep models on both recall and F-score.
4. **Naive Bayes and KNN are more sensitive to overlap/noise** in dynamic behavior features, resulting in lower overall reliability.
5. **Compared to the paper, implementation scores are slightly lower but close**, which is realistic for different data composition, preprocessing, and experimental environment.

5.4 Key Observations from Paper

1. **XRan outperforms all baselines** on the combined dataset, achieving 99.4% TPR with only 1.0% FPR.
2. **Sequence combination is crucial**: Using only API calls yields 85-88% accuracy. Adding DLLs and Mutexes in sequence form (not frequency) improves accuracy to 98-100%.
3. **Deep learning > ML for complex datasets**: For RD1 (most similar to benign), CNN/LSTM significantly outperform ML methods. For RD2/RD3 (more distinct), even Random Forest achieves near-perfect scores.
4. **DLLs and Mutexes are disproportionately important**: Despite occupying only 20 of 520 positions, DLL and Mutex features contribute more to the model's decisions than the 500 API call positions (per SHAP analysis).
5. **Ransomware vs. Malware is harder than Ransomware vs. Benign**: The malicious behavior overlap between ransomware and general malware makes discrimination more challenging.

6. Observations and Conclusions

6.1 Strengths of XRan

- **Multi-perspective feature fusion**: Combining API calls, DLLs, and Mutexes as concatenated sequences preserves both individual feature patterns and cross-feature correlations.
- **Hierarchical feature extraction**: The 2-layer CNN captures both low-level and high-level patterns through hierarchical convolution.
- **Explainability**: LIME and SHAP integration provides actionable insights for security analysts, identifying specific APIs, DLLs, and Mutexes that characterize ransomware behavior.

6.2 Limitations

- **Dynamic analysis overhead**: Cuckoo Sandbox analysis averages 155.3 seconds per sample, making real-time deployment challenging.
- **Evasion potential**: Sophisticated ransomware may detect sandbox environments and modify behavior.

- **Dataset dependency**: Performance varies across datasets (RD1 is harder to classify than RD2/RD3), suggesting sensitivity to the feature space overlap between classes.
- **Fixed sequence lengths**: The truncation of API calls to 500 may lose late-stage behavioral information for long-running samples.

6.3 Conclusions

XRan demonstrates that combining multiple dynamic analysis features in sequence form with a deep CNN architecture achieves state-of-the-art ransomware detection. The integration of XAI models provides valuable insights:

- Ransomware-indicative features: `NtAllocateVirtualMemory`, `RegOpenKeyExW`, `Advapi32.dll`, `Crypt32.dll`, `ZonesCounterMutex`
- Benign-indicative features: `LoadStringA/W`, `Kernel32.dll`, `Version.dll`
- DLLs and Mutexes provide stronger discriminative signals per feature than API calls

The study validates that multi-view dynamic analysis combined with deep learning and explainability is a promising direction for next-generation ransomware detection systems.

References (Assigned Papers)

1. Gulmez, S., Gorgulu Kakisim, A., & Sogukpinar, I. (2024). XRan: Explainable deep learning-based ransomware detection using dynamic analysis. *Computers & Security*, 139, 103703.
2. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. *KDD*.
3. Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *NeurIPS*.